

MC Core — Internal Block Field Definitions

Post-CDC: Buffers, Bank FSM Table, Timer Scheme

Field-level spec — scheduler behaviour deferred

Scope	Everything after the two Async FIFOs, before Cmd Scheduler logic
Covers	Req/Resp FIFO packet formats, Write Data Buffer, Write/Read Request Buffers, Bank FSM T
Timer model	Absolute <code>next_legal_cycle</code> registers, not live countdown counters
Not covered	Scheduler arbitration algorithm, page policy decision logic (defined elsewhere)

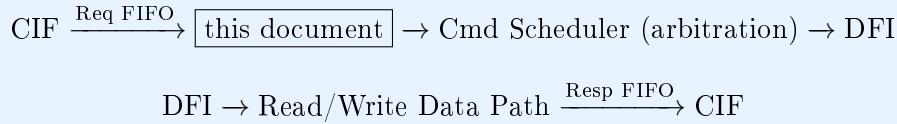
Contents

1	Scope and Position in Pipeline	2
2	Async FIFO Packet Formats	2
2.1	Request FIFO (CIF → MC Core)	2
2.2	Response FIFO (MC Core → CIF)	2
2.3	Resp FIFO Flow Control — Valid-Only	3
2.4	Req FIFO Flow Control	3
3	Global Cycle Counter	3
4	Write Data Buffer	4
5	Write Request Buffer	4
5.1	Field Definitions	5
5.2	Allocation and Retirement	5
6	Read Request Buffer	6
6.1	Field Definitions	6
6.2	Allocation and Retirement	6
7	Bank FSM Table	7
7.1	Per-Bank Row Fields	7
7.2	Per-Bank Timing Eligibility Fields	8
7.3	Global (Cross-Bank) Timing Fields	9
7.4	Resp FIFO Availability Gate	9
7.5	Update Timing — When Fields Get Written	10
8	Summary: Field Count and Storage Sizing	11

Scope and Position in Pipeline

This document picks up immediately after the two Async FIFO CDC crossings and stops at the boundary of the Cmd Scheduler’s arbitration logic. It defines the *data structures* the scheduler will operate on, not the scheduler’s decision algorithm itself.

Pipeline position:



Async FIFO Packet Formats

Two unidirectional FIFOs cross the AXI clock $\leftrightarrow$ MC clock boundary. Each carries a single polymorphic packet format distinguished by a type field.

Request FIFO (CIF $\rightarrow$ MC Core)

Field	Width	Description
<code>req_type</code>	1b	0 = <code>req_read</code> , 1 = <code>req_write</code>
<code>tag</code>	varies (AXI ID width)	AXI AWID (write) or ARID (read). Carried through unchanged — this is the only identifier the resp path needs to route data back to the correct AXI master/ID.
<code>addr</code>	32b	Byte address. Already past Burst Splitter and Address Translator — but for this doc we treat <code>addr</code> as the raw 32b field; the bank/row/col split happens when this entry is loaded into the Write/Read Request Buffer (§4, §5).
<code>data</code>	512b	Write only. 64 B write data (one full BL16 burst worth). Zero/don’t-care for <code>req_read</code> .
<code>mask</code>	64b	Write only. Byte-level write mask (1 bit per byte of <code>data</code>). All-0 = full write, any 1 bit = <code>wr_partial</code> candidate. Don’t-care for <code>req_read</code> .

`req_read` packet effectively only uses {`req_type`=0, `tag`, `addr`} — the 512b `data` and 64b `mask` fields are simply unused for reads. Whether this is implemented as a fixed-width packet with unused bits, or a variable-width packet via a separate narrow read-request FIFO lane, is an RTL packing decision outside this doc’s scope — functionally identical either way.

Response FIFO (MC Core $\rightarrow$ CIF)

Field	Width	Description
<code>resp_type</code>	2b	00 = <code>RD_DATA</code> , 01 = <code>WR_ACK</code> , 10 = <code>ERR</code> . (11 reserved.)

Field	Width	Description
<code>tag</code>	varies	Same AXI ID carried from the original request. CIF uses this to route the response to the correct ROB slot (read) or generate <code>BRESP</code> (write).
<code>data</code>	512b	RD_DATA only. 64 B read data. Zero/don't-care for <code>WR_ACK</code> and <code>ERR</code> .
<code>status</code>	4b	Status/error code. For <code>WR_ACK</code> : normally 0 (OKAY). For <code>ERR</code> : encodes which error (CRC fail, alert, timeout, etc. — exact encoding TBD).

Resp FIFO Flow Control — Valid-Only

Design decision: the Resp FIFO presents `valid` to the CIF side only. **No `ready` signal is offered** — the CIF must always be able to accept a response the instant `valid` is asserted.

Why: read data, once it starts arriving from DFI, cannot be held back — `dfi_rddata_valid` is a fire-and-forget signal from the PHY with a fixed latency from the RD command. If the Resp FIFO could assert backpressure that propagates to “don’t issue this RD yet,” that backpressure would have to reach all the way back through the Cmd Scheduler *before* the RD is issued — adding a dependency between CIF drain rate and DRAM command issue rate that this design explicitly avoids.

Consequence: the MC side must guarantee the Resp FIFO never overflows *before* issuing a command that will produce a response. This is enforced by `gate_resp_fifo_avail` (§6.3) — a new gate checked by the scheduler *before* issuing any RD (and before issuing any WR if `WR_ACK` responses are mandatory).

Req FIFO Flow Control

Open: valid/ready vs. valid/credit for the Req FIFO has not been decided. This affects how `req_write` backpressure propagates to the AXI `AWREADY`/`WREADY` signals at the CIF boundary. Both schemes are compatible with everything else in this document — the Write/Read Request Buffers (§4, §5) are the actual backpressure point regardless (a new request can’t be accepted into the Req FIFO’s consumer side if both request buffers are full).

Global Cycle Counter

A single free-running counter, `global_cycle[31:0]`, incremented every MC clock cycle. Never resets except on `SOFT_RESET`. This is the reference against which every `next_legal_cycle` field (§4–6) is compared.

Field	Width	Description
<code>global_cycle</code>	32b	Free-running. Increments by 1 every MC clock. At 32b, wraps after 2^{32} cycles — at 200 MHz that's ≈ 21.5 seconds. Wraparound handling is an open implementation detail (e.g. all comparisons done modulo, or periodic rebasing).

Why one global counter instead of per-bank counters? Every `next_legal_cycle` register is computed once, at issue time, as `global_cycle + delay`. Checking eligibility later is a single comparator: `global_cycle >= next_legal_cycle[bank][cmd]`. This is the mechanism that replaces “200 different counters” with one shared counter plus a bank of comparators — the arithmetic (the addition) only happens at issue time, which is rare (once per command); the comparison (cheap) happens every cycle for every still-pending constraint.

Write Data Buffer

A 32-entry SRAM, index-addressed. Stores the 512b write data payload separately from the Write Request Buffer's control fields, so the control-field table (§5) stays narrow and fast to scan.

Field	Width	Description
<code>data[511:0]</code>	512b	The 64 B write payload from <code>req_write</code> .
<code>mask[63:0]</code>	64b	Byte-level write mask, copied alongside data.

Property	Value	Notes
Depth	32 entries	Matches Write Request Buffer depth 1:1.
Addressing	Index-addressed	Index = the slot number in the Write Request Buffer (<code>data_buf_idx</code> field, §5). Not content-addressed, not FIFO-ordered.
Entry size	576b (512+64)	Data + mask packed together per entry.
Total size	$32 \times 576b = 18\,432b \approx 2.25\text{ KB}$	SRAM macro sizing reference.

Why separate from the Write Request Buffer? The 512b data field dominates entry size. Keeping it in a separate SRAM means the Write Request Buffer (§5) — which the scheduler scans every cycle across all 32 entries — stays narrow (control fields only, no data). The SRAM is only read once, at the moment the WR command is actually issued and data needs to go on `dfi_wrdata`.

Write Request Buffer

32 entries. Holds control/address fields for pending writes. The scheduler scans **all valid entries every cycle** (CAM-like, transparent) — not a FIFO, no head/tail pointer for scheduling purposes.

Field Definitions

Field	Width	Description
<code>valid</code>	1b	Entry occupied. Set on allocation, cleared on retirement (after <code>WR_ACK</code> sent).
<code>axi_tag</code>	varies	Copied from <code>req_write.tag</code> (AWID). Needed to generate the <code>WR_ACK</code> response with the correct routing tag.
<code>bank_group</code>	3b	From Address Translator output.
<code>bank</code>	2b	From Address Translator output.
<code>row</code>	17b	From Address Translator output.
<code>column</code>	10b	From Address Translator output.
<code>data_buf_idx</code>	5b	Index into the Write Data Buffer SRAM (§4). Set equal to this entry’s own slot number at allocation (1:1 mapping, so this field is technically redundant with the entry’s own address — kept explicit for clarity / in case 1:1 mapping changes later).
<code>wr_partial</code>	1b	Set if <code>mask</code> $\neq$ 0 (any byte masked). Signals the eventual WR command needs <code>WR_PARTIAL</code> encoding (triggers DRAM-internal RMW).
<code>status</code>	2b	00=PENDING (not yet issued to DFI), 01=ISSUED (WR command sent, awaiting tWR completion), 10=DONE (tWR satisfied, ready to retire/ack), 11=ERROR.
<code>issue_cycle</code>	32b	<code>global_cycle</code> value at the moment the WR command was issued. Used to compute <code>next_legal_cycle</code> entries in the Bank FSM Table (§6) at issue time — <i>not</i> re-read after that.

Why “all 32 entries, transparent” instead of head-of-queue only? Bank-group-aware reordering (per the `arb_com_sel` reference doc) needs the scheduler to find, e.g., “any pending write to a *different* bank group than the last-issued write,” regardless of allocation order. A FIFO-ordered view would only ever offer the oldest entry — exactly the case that triggers the `tCCD_L_WR=32 nCK` penalty when the oldest entry happens to share a BG with the just-issued command. Full visibility lets the scheduler pick a *younger* entry in a different BG instead, while the oldest entry simply waits one more cycle (its age counter increments, eventually forcing it via age escalation if this keeps happening).

Allocation and Retirement

Event	Field Updates
Allocation (Req FIFO pop, <code>req_write</code>)	Find first <code>valid=0</code> slot. Write <code>axi_tag</code> , <code>bank_group/bank/row/column</code> (post-translation), <code>wr_partial</code> , <code>data_buf_idx</code> =own slot index. Set <code>valid=1</code> , <code>status</code> =PENDING. Write Data Buffer slot <code>data_buf_idx</code> simultaneously loaded with <code>data/mask</code> .

Event	Updates
Issue (WR command sent to DFI)	<code>status</code> → ISSUED. <code>issue_cycle</code> = <code>global_cycle</code> . Bank FSM Table's relevant <code>next_legal_cycle</code> entries (§6.4) updated using <code>issue_cycle</code> + appropriate delay.
tWR satisfied	<code>status</code> → DONE. (Detected via Bank FSM Table's <code>next_act/next_pre</code> for this bank reaching <code>global_cycle</code> — see §6.)
Retirement (WR_ACK sent on Resp FIFO)	<code>valid</code> → 0. Slot available for new allocation.

Read Request Buffer

Same structure as the Write Request Buffer, minus the data fields (no data exists yet for a read), plus a Resp FIFO slot reservation field.

Field Definitions

Field	Width	Description
<code>valid</code>	1b	Entry occupied.
<code>axi_tag</code>	varies	Copied from <code>req_read.tag</code> (ARID).
<code>bank_group</code>	3b	From Address Translator output.
<code>bank</code>	2b	From Address Translator output.
<code>row</code>	17b	From Address Translator output.
<code>column</code>	10b	From Address Translator output.
<code>resp_slot</code>	5b	Reserved slot/credit in the Resp FIFO for this read's eventual RD_DATA packet. Allocated <i>before</i> the RD command is issued (see <code>gate_resp_fifo_avail</code> , §6.3 – the RD cannot be issued unless a slot can be reserved).
<code>status</code>	2b	00=PENDING, 01=ISSUED (RD command sent, awaiting <code>dfi_rddata_valid</code>), 10=DONE (data captured, ready to send on Resp FIFO), 11=ERROR.
<code>issue_cycle</code>	32b	<code>global_cycle</code> value at RD issue. Used to compute <code>next_legal_cycle</code> entries in the Bank FSM Table (§6).

Why no data field here? Unlike the Write Request Buffer, a read has no payload to store on the request side — the 512b that eventually needs to travel back to CIF arrives later, directly from the Read Data Path's DFI capture FIFO, tagged with this entry's `axi_tag` and routed via `resp_slot`. The Read Data Path's capture FIFO is the “read data buffer” equivalent of §4 — outside this doc's scope (belongs to the Read Data Path sub-block).

Allocation and Retirement

Event	Field Updates
Allocation (Req FIFO pop, req_read)	Find first <code>valid=0</code> slot. Write <code>axi_tag</code> , <code>bank_group/bank/row/column</code> . Set <code>valid=1</code> , <code>status=PENDING</code> . <code>resp_slot</code> not yet assigned.
Resp slot reservation (pre-issue)	Scheduler checks Resp FIFO has a free slot (<code>gate_resp_fifo_avail</code> , §6.3). If yes, reserve it into <code>resp_slot</code> . This must succeed <i>before</i> the RD command is allowed to issue.
Issue (RD command sent to DFI)	<code>status</code> → ISSUED. <code>issue_cycle</code> = <code>global_cycle</code> . Bank FSM Table updated (§6.4).
Data capture (Read Data Path, dfi_rddata_valid)	<code>status</code> → DONE. Read Data Path forwards {data, <code>axi_tag</code> , <code>resp_slot</code> } toward Resp FIFO.
Retirement (RD_DATA sent on Resp FIFO)	<code>valid</code> → 0. <code>resp_slot</code> released.

Bank FSM Table

One row per physical bank. **16 rows total** (2 BG × 4 banks × 2). Each row is a fixed-width record. The scheduler reads this table every cycle to determine which banks are eligible for which command types — a **GO/NO-GO** lookup, not a state-machine traversal.

Per-Bank Row Fields

Field	Width	Description
<code>state</code>	4b	Current bank state. Encoding: 0000=IDLE, 0001=ACTIVATING, 0010=ACTIVE, 0011=READING, 0100=WRITING, 0101=PRECHARGING, 0110=REFRESHING_AB, 0111=REFRESHING_SB, 1000=POWER_DOWN, 1001=SELF_REFRESH, 1010=RFM_ACTIVE.
<code>row_open</code>	17b	The row address currently latched in this bank's sense amplifiers. Valid only when <code>state</code> ∈ {ACTIVE, READING, WRITING}. Compared against an incoming request's <code>row</code> field for hit/miss decision.
<code>open_pending</code>	1b	GO/NO-GO summary bit: 1 if this bank currently has an outstanding read or write request in the Read/Write Request Buffers targeting it. Lets the scheduler skip banks with nothing to do without scanning both request buffers per bank per cycle. (Maintained incrementally: set on allocation targeting this bank, cleared when no remaining buffer entries target it.)

Field	Width	Description
<code>pre_pending</code>	1b	GO/NO-GO: 1 if a PRE is needed before the next CAS can issue (i.e. <code>open_pending</code> =1 AND the pending request's row $\neq$ <code>row_open</code> AND <code>state</code> =ACTIVE). This is the “row miss” flag.
<code>act_pending</code>	1b	GO/NO-GO: 1 if an ACT is needed before the next CAS (i.e. <code>open_pending</code> =1 AND <code>state</code> =IDLE, or <code>pre_pending</code> =1 and the PRE has already completed).
<code>wr_pending</code>	1b	GO/NO-GO: 1 if the pending request for this bank (per <code>open_pending</code>) is specifically a write .
<code>rd_pending</code>	1b	GO/NO-GO: 1 if the pending request for this bank is specifically a read . (<code>wr_pending</code> and <code>rd_pending</code> are mutually exclusive per the scheduler's one-bank-one-target-at-a-time model — if both a pending read <i>and</i> a pending write target the same bank, the scheduler's RD/WR mode arbitration, <i>defined elsewhere</i> , picks which one sets these bits.)
<code>ref_pending</code>	1b	GO/NO-GO: 1 if the Refresh Scheduler has marked this bank as due for REFsb (or this bank is part of a pending REFab affecting all banks).

This is the “dictionary” format you described: for bank 0, a row might read `state`=ACTIVE, `pre_pending`=0, `act_pending`=0, `wr_pending`=0, `rd_pending`=1 — i.e. “bank 0 is active with a row open, no PRE/ACT needed, and there's a pending read targeting it.” The scheduler's job (defined elsewhere) is to scan all 16 rows and pick which bank(s) get serviced this cycle based on these flags plus the timing-eligibility fields below.

Per-Bank Timing Eligibility Fields

These are the `next_legal_cycle` registers — absolute `global_cycle` values, not countdowns.

Field	Width	Description
<code>next_act</code>	32b	Earliest <code>global_cycle</code> at which an ACT may issue to this bank. Loaded from: (a) PRE issue + tRP, if <code>state</code> =PRECHARGING; or (b) the cross-bank tRRD_L/tRRD_S/tFAW checks (§6.5) — whichever is later.
<code>next_pre</code>	32b	Earliest <code>global_cycle</code> at which a PRE may issue to this bank. Loaded from: ACT issue + tRAS (if last command was ACT), or RD issue + tRTP, or WR data-end + tWR — whichever applies based on <code>state</code> .

Field	Width	Description
<code>next_cas</code>	32b	Earliest global_cycle at which the <i>next</i> RD or WR may issue to this bank, considering only this bank's own timing (tRCD if just activated). Cross-bank tCCD_L/tCCD_S/tWTR/tRTW checks are separate (§6.5) since they're not per-bank quantities.
<code>next_ref</code>	32b	Earliest global_cycle at which a REFsb may issue to this bank (tRfCsb from the last REFsb to this bank), or at which this bank becomes available again after a REFab/REFsb currently in progress (tRfC1/tRfCsb from issue).

Global (Cross-Bank) Timing Fields

Not per-bank — these apply across all banks and live alongside the per-bank table as shared registers.

Field	Width	Description
<code>next_act_same_bg[3]</code>	32b × #BG	Per-BG: earliest global_cycle for the next ACT in this BG (tRRD_L from the last ACT in this BG). Indexed by bank group (3b → up to 8 entries).
<code>next_act_any</code>	32b	Earliest global_cycle for the next ACT to <i>any</i> bank (tRRD_S from the last ACT anywhere).
<code>faw_window[3:0]</code>	32b × 4	Ring buffer of the 4 most recent ACT global_cycle timestamps (any bank). An ACT is blocked if global_cycle – <code>faw_window[oldest]</code> < tFAW.
<code>next_cas_same_bg[3]</code>	32b × #BG	Per-BG: earliest global_cycle for the next CAS (RD or WR) in this BG (tCCD_L / tCCD_L_WR / tCCD_L_WR2 from the last CAS in this BG, depending on cmd types involved).
<code>next_cas_any</code>	32b	Earliest global_cycle for the next CAS to <i>any</i> BG (tCCD_S from the last CAS anywhere).
<code>next_rd_after_wr[3]</code>	32b × #BG	Per-BG: earliest global_cycle a RD may issue in this BG (tWTR_L from last WR data-end in this BG).
<code>next_rd_after_wr_</code>	32b	Earliest global_cycle a RD may issue to any BG (tWTR_S from last WR data-end anywhere).
<code>next_wr_after_rd</code>	32b	Earliest global_cycle a WR may issue (tRTW from last RD command, applies globally per spec formula $RL+BL/2-WL+2$).

Resp FIFO Availability Gate

Field	Width	Description
<code>gate_resp_fifo_av</code>	1b	Combinational. 1 if the Resp FIFO has at least one free slot <i>not already reserved</i> by another in-flight read's <code>resp_slot</code> . Must be 1 for any RD to be eligible for issue — this is checked alongside the timing-eligibility fields above, as an additional AND term in the overall go/no-go for any <code>rd_pending=1</code> bank.

Where this plugs in: a bank with `rd_pending=1`, `state=ACTIVE`, `pre_pending=0`, `act_pending=0`, and `global_cycle ≥` all relevant `next_*` fields is normally “ready to issue RD.” `gate_resp_fifo_avail=0` overrides this to NO-GO regardless — the bank stays eligible (its timing windows don’t close), it just doesn’t get picked *this cycle*. As soon as the Resp FIFO drains (CIF consumes a RD_DATA), the gate flips back to 1 and the bank becomes pickable again with no re-computation of any timing field.

Update Timing — When Fields Get Written

Event	Fields Updated	New Value
ACT issued to bank b , BG g	<code>state</code> →ACTIVATING, <code>row_open</code> →req.row, <code>next_cas</code> , <code>next_act_same_bg[g]</code> , <code>next_act_any</code> , <code>faw_window</code>	<code>next_cas</code> = <code>global_cycle</code> +tRCD. <code>next_act_same_bg[g]</code> = <code>global_cycle</code> +tRRD_L. <code>next_act_any</code> = <code>global_cycle</code> +tRRD_S. <code>faw_window</code> shifts in <code>global_cycle</code> .
PRE issued to bank b	<code>state</code> →PRECHARGING, <code>next_act</code>	<code>next_act</code> = <code>global_cycle</code> +tRP.
tRCD elapsed (<code>global_cycle ≥ next_act</code> was ACTIVATING)	<code>state</code> →ACTIVE	— (no new timer; <code>next_cas</code> already holds the value that just triggered this transition).
RD issued to bank b , BG g	<code>state</code> →READING, <code>next_pre</code> , <code>next_cas_same_bg[g]</code> , <code>next_cas_any</code> , <code>next_wr_after_rd</code>	<code>next_pre</code> = <code>global_cycle</code> +tRTP. <code>next_cas_same_bg[g]</code> = <code>global_cycle</code> +tCCD_L. <code>next_cas_any</code> = <code>global_cycle</code> +tCCD_S. <code>next_wr_after_rd</code> = <code>global_cycle</code> +tRTW.
WR issued to bank b , BG g (data-end at issue+CWL+BL/2)	<code>state</code> →WRITING, <code>next_pre</code> , <code>next_cas_same_bg[g]</code> , <code>next_cas_any</code> , <code>next_rd_after_wr[g]</code> , <code>next_rd_after_wr_any</code>	<code>next_pre</code> = <code>global_cycle</code> +CWL+BL/2+tWR. <code>next_cas_same_bg[g]</code> = <code>global_cycle</code> +tCCD_L_WR (or _WR2 if next CAS in this BG has <code>wr_partial=1</code>). <code>next_cas_any</code> = <code>global_cycle</code> +tCCD_S. <code>next_rd_after_wr[g]</code> = (issue+CWL+BL/2)+tWTR_L. <code>next_rd_after_wr_any</code> = (issue+CWL+BL/2)+tWTR_S.

Event	Fields	New Value
tRTP/tWR elapsed (<code>global_cycle</code> ≥ <code>next_pre</code>) was READ-ING/WRITING)	<code>state</code> → ACTIVE (open page) or <code>ap</code> = 1 (closed page / RDA / WRA)	If →PRECHARGING: <code>next_act</code> = <code>global_cycle</code> + tRP (i.e. the hidden-precharge’s own tRP starts now).
tRP elapsed (<code>global_cycle</code> ≥ <code>next_pre</code>) was PRECHARG-ING)	<code>state</code> → IDLE	—
REFab issued (all banks)	all 16 rows: <code>state</code> → REFRESHING_AB <code>next_act</code>	<code>next_act</code> = <code>global_cycle</code> + tRFC1 (all banks simultaneously).
REFsb issued to bank <i>b</i>	bank <i>b</i> row: <code>state</code> → REFRESHING_SB <code>next_act</code> , <code>next_ref</code>	<code>next_act</code> = <code>next_ref</code> = <code>global_cycle</code> + tRFCsb. <i>Other banks’ rows unaffected.</i>
tRFC1/tRFCsb elapsed (was REFRESH-ING_AB/SB)	<code>state</code> → IDLE	—

Why `next_pre` for WRITING uses $CWL + BL/2 + tWR$ as one lump sum, computed at issue time, rather than “data-end timestamp” + “tWR” as two separate fields? Both are mathematically equivalent ($global_cycle_{issue} + CWL + BL/2 + tWR = \text{data-end} + tWR$ since $\text{data-end} = \text{issue} + CWL + BL/2$). Storing the single combined sum means **one** field and **one** comparator does the job of what would otherwise be two fields and an intermediate “has data-end happened yet” check. This is the same “absolute future cycle, one comparator” principle applied consistently everywhere in this table.

Summary: Field Count and Storage Sizing

Structure	Total Bits	Notes
Write Data Buffer (32 entries × 576b)	18 432b	SRAM macro
Write Request Buffer (32 × ≈40b)	≈1280b	Register file / small SRAM
Read Request Buffer (32 × ≈39b)	≈1248b	Register file / small SRAM
Bank FSM Table, per-bank fields (16 × (4+17+5+4×32b))	16 × 154 = 2464b	<code>state</code> + <code>row_open</code> + 5 go/no-go bits + 4 × <code>next_*</code>

Structure	Bits	Notes
Bank FSM Table, global fields (≈ 8 BG entries $\times$ 4 fields $\times$ 32b + misc)	≈ 1200 b	<code>next_act_same_bg</code> , <code>next_cas_same_bg</code> , <code>next_rd_after_wr[bg]</code> $\times$ 8 BGs, plus <code>faw_window(4$\times$32b)</code> , <code>next_act_any</code> , <code>next_cas_any</code> , <code>next_rd_after_wr_any</code> , <code>next_wr_after_rd</code>
<code>global_cycle</code>	32b	Single free-running counter
<code>gate_resp_fifo_avail</code>	1b	Combinational, not stored
Approx. total (excl. Write Data Buffer SRAM)	≈ 6225 b $\approx$ 778 B	Comparators: ≈ 16 banks $\times$ 4 <code>next_*</code> + global fields ≈ 100 comparators total, all 32b-vs-32b, all running every cycle.

Sanity check against “200 counters”: a naive per-constraint countdown design would need one decrementing counter *per (bank, constraint)* pair — 16 banks $\times$ (tRCD, tRAS, tRP, tWR, tRTP, tCCD_L, tCCD_S, tWTR_L, tWTR_S, tRRD_L, tRRD_S, tFAW, tRFC...) $\approx 16 \times 13 \approx 200+$ live decrementers, each needing its own subtractor every cycle. The `next_legal_cycle` scheme above collapses this to ≈ 100 **static comparators** against **one** shared incrementing counter — comparators are cheaper than subtractors, and the “13 constraints” largely collapse into ≤ 4 per-bank fields plus a handful of shared global fields, because many constraints (tRCD, tRAS feeding tRP, etc.) chain into the *same* `next_*` field rather than needing independent storage.